

Tuning Workflow Management Systems for shared HPC resources

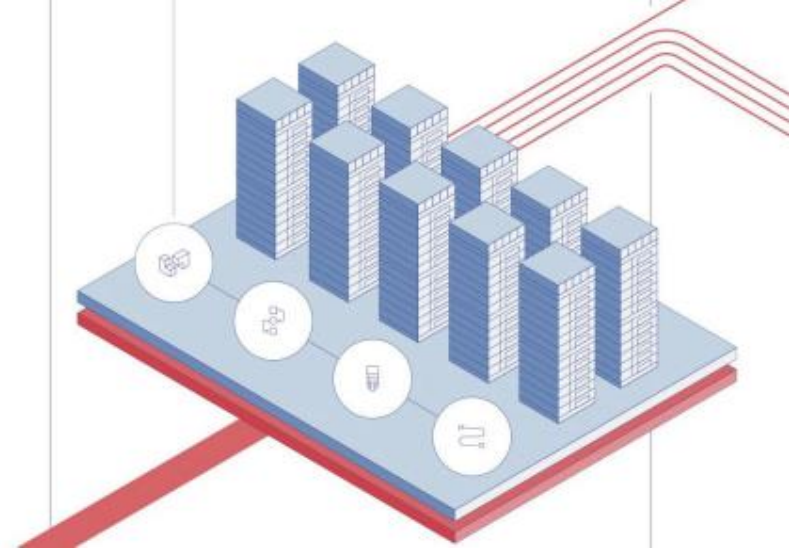
Nil Mu, HPC System Admin @ ASU RTO Research Computing

Content

- **Overview**
- **Popular Options**
- **Basic Pipeline Structure**
- **Brief Coding Examples**
- **Challenges**
- **Best Practices**
- **Benchmarking**

Keywords

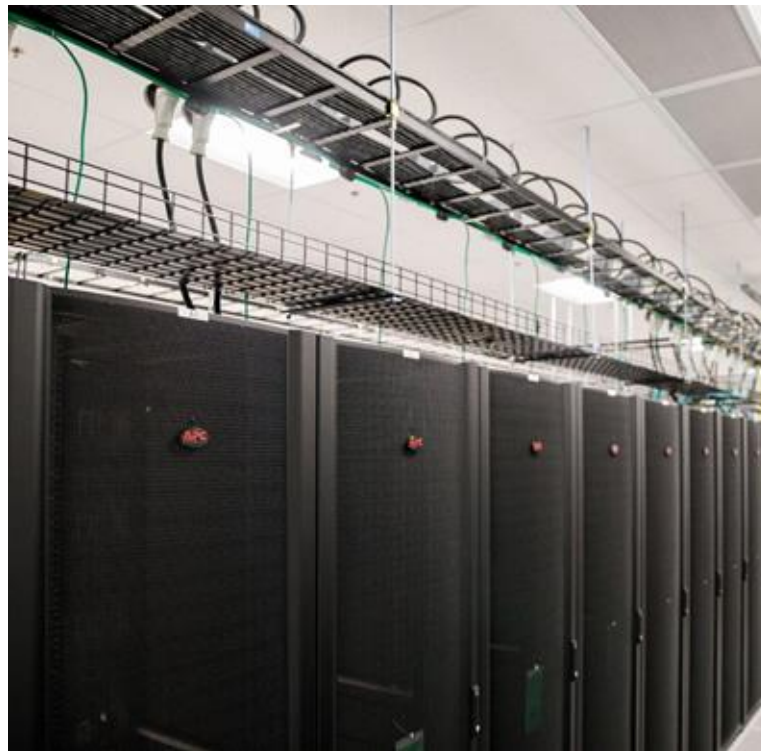
- **Supercomputer**
 - **High-Performance Computing cluster**
- **Slurm**
 - **Job scheduler system**
- **Fairshare**
 - **Computational resources consumption**
- **Workflow**
 - **Multi-steps computational pipeline**



ASU Supercomputers

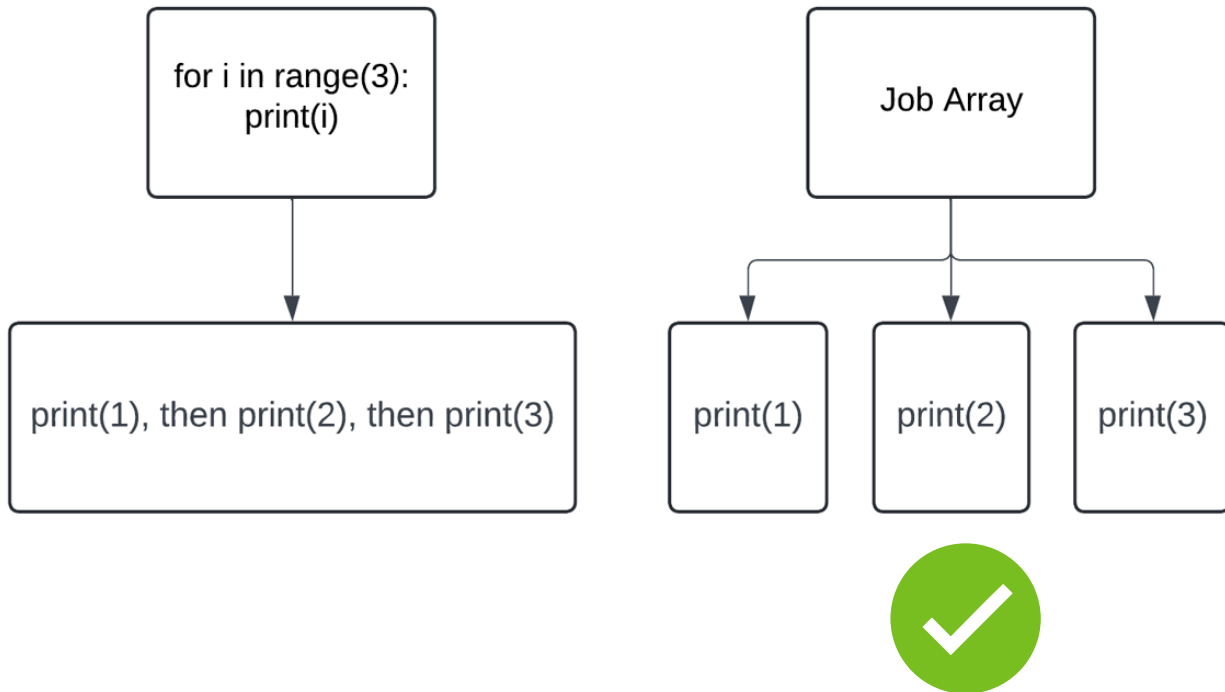
- Sol
 - Flagship, Top500
- Phoneix
 - CPU-intensive
- Aloe
 - HIPAA-compliant
- New AI cluster!

docs.rc.asu.edu



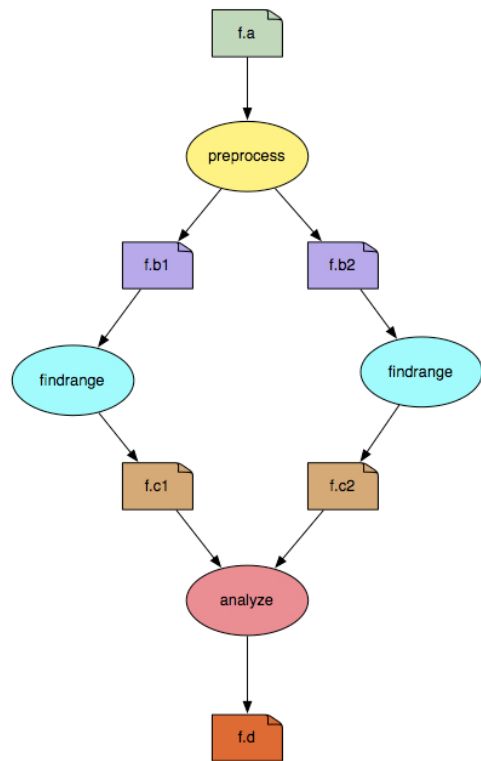
How to Compute Efficiently?

- Main goal: Run jobs in parallel, not serial

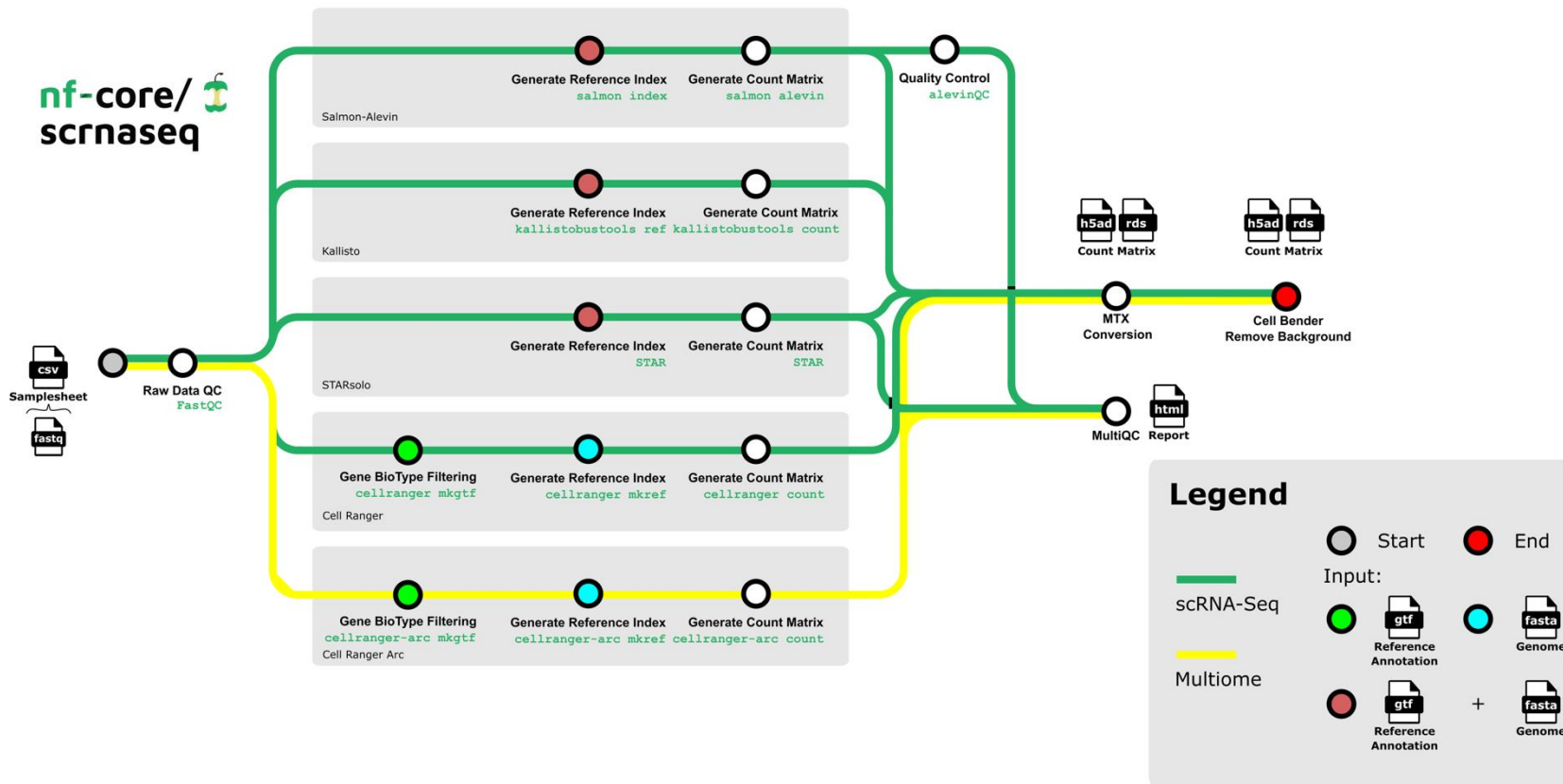


Workflow Management System (WMS)

- Main goal: Run jobs in parallel, not serial
- DIY array vs. Slurm job array
 - Fairshare score & Pending time
- Slurm job array vs. WMS
 - Linear workflow
 - $a \rightarrow b \rightarrow c \rightarrow d$
 - Complicated branched workflow

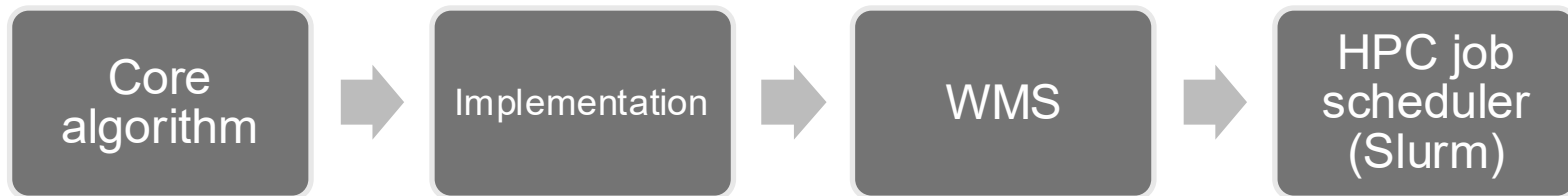


Complicated Pipeline Example



Workflow Management System (WMS)

- Main goal: Run jobs in parallel, not serial
- WMS: Communicates between your codes and the scheduler
- DSL (Domain-Specific Language): WMS syntax
- DAG (Directed Acyclic Graph): the task-dependency graph the WMS builds and walks



1. I/O
2. Resources
3. Tracking
4. Errors ...

Popular WMS

2026 Spring



Snakemake

v9

Pythonic

10-1K jobs

parse-time DAG

timestamp-based



Nextflow

v26

Groovy, Java

100K jobs

nf-core ecosystem, lint

job array, cloud, container

run-time DAG, hashed



Pegasus

v5

Python, Java

1M Jobs

multi-site



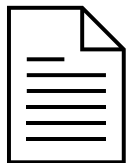
More

362 and growing

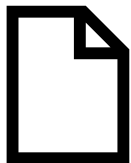
Basic Pipeline Structure



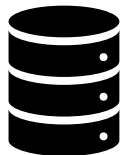
Step description files
For actual computation



Resources configuration files
For the child jobs



Sbatch job submission script
for the main job



Input & output directories
containers, environments



Main Job

long time,
less CPU,
less memory

Child Jobs

compute

child job 1

child job 2

...

Snakemake v9 Example

Snakefile: align.smk

```
rule all:
    input:
        expand("results/{sample}.bam",
        sample=SAMPLES)

rule trim:
    input: "raw/{sample}.fq"
    output: temp("trimmed/{sample}.fq")
    conda: "/path/to/conda/envs/alignment"
    shell: "fastp -i {input} -o {output} -
q 20 -u 30"
```

```
#!/bin/bash
#SBATCH -J snake_ctrl
#SBATCH -t 12:00:00
#SBATCH -N 1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH -o snake_ctrl.out
#SBATCH -e snake_ctrl.err
```

module load snakemake

```
snakemake --profile profiles/slurm \
--use-conda --conda-frontend mamba \
--jobs 200 \
--rerun-incomplete \
-s workflows/align.smk
```

main.sh

tmux

```
executor: slurm
jobs: 200
default-resources:
    time: "01:00:00"
    mem_mb: 4000
    threads: 1
cluster: "sbatch -A my_account -p general -t {resources.time} -c {threads} --mem={resources.mem_mb}"
```

profiles/slurm/config.yaml

Nextflow v26 Example

nextflow.config

```
params.samples = "samples/*.fq"
process TRIM {
    input:
        path fq
    output:
        path "${fq.baseName}.trim.fq" into trimmed_ch
    conda "/path/to/conda/envs/alignment"
    script:
        """
        fastp -i $fq -o ${fq.baseName}.trim.fq -q 20
        """
}
```

align.nf

```
#!/bin/bash
#SBATCH -c 1
...
module load nextflow

nextflow run main.nf -c site.config -profile slurm
```

main.sh

```
// Nextflow config for lastz jobs
executor {
    queueSize = 10000
    retry.maxAttempt = 3
    killBatchSize = 1000000
}

process {
    executor = 'slurm'
    memory = { 4.GB * task.attempt }
    time = { 1.hour * task.attempt }
    queue = 'public'
    cpus = 1
    array = 2000
    maxRetries = 5
    errorStrategy = 'retry'
    maxErrors = '-1'
    clusterOptions = '--signal=USR2@180'
}
```

Challenges

Challenges Overview

Admins

- **Scheduler interaction**
- **Fair-share impact**
- **Resource efficiency**
- **Cluster-wide utilization**

Users

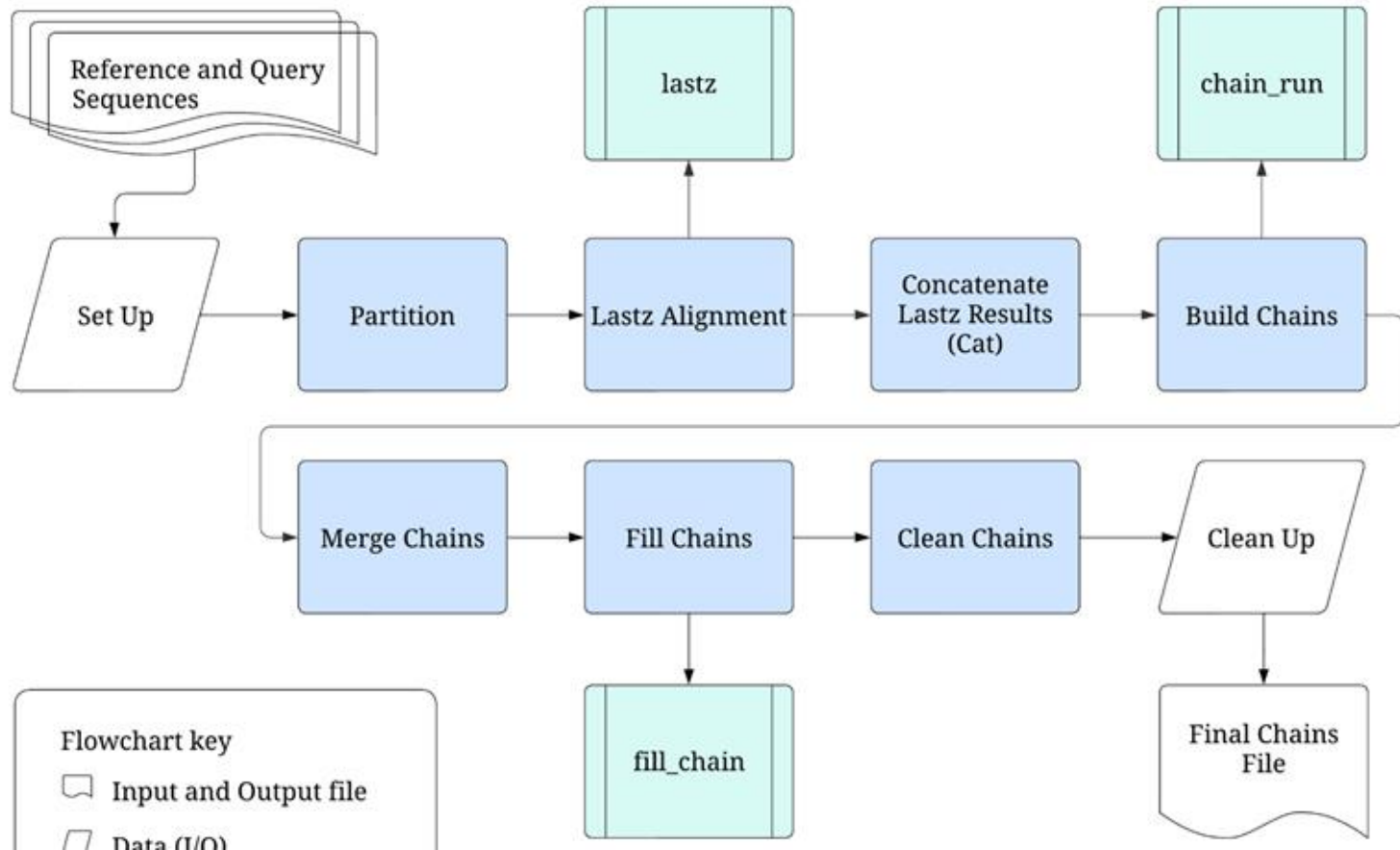
- **Over-provisioned memory & time**
- **Runaway submissions**
- **Poor locality awareness**

Earth Biogenome Project (EBP)

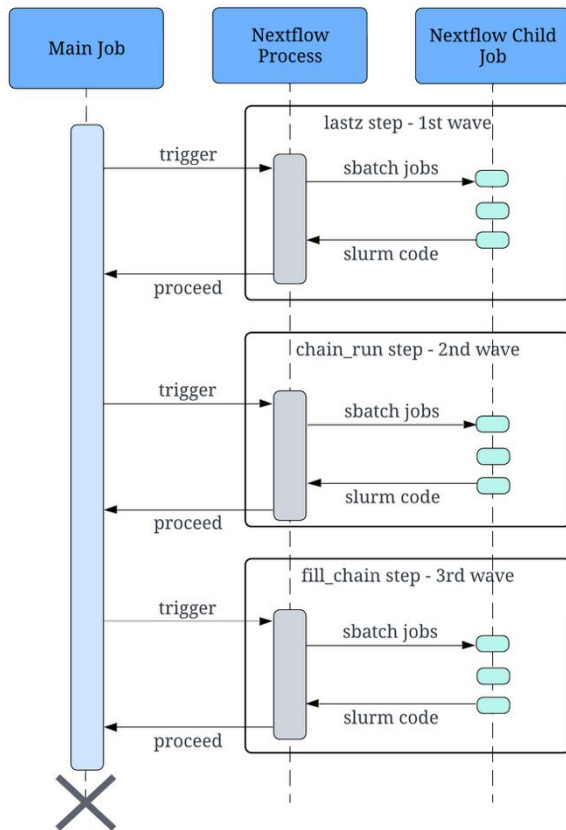
- Representative eukaryotic species
- \$4.7 Billion
- 12.7% (10k)
- Compare long strings



make_lastz_chains

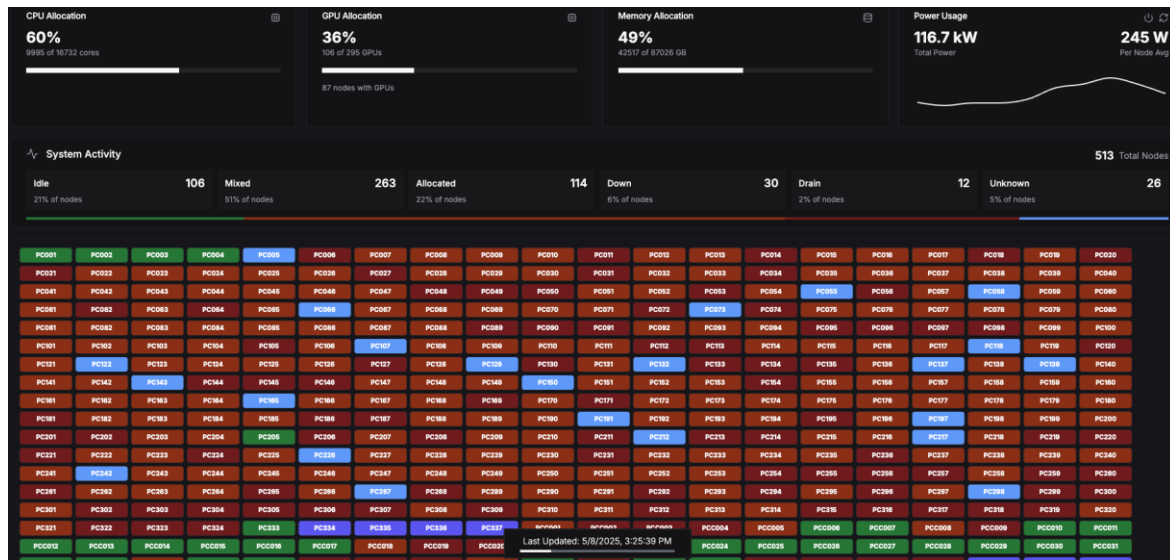


Challenge #1: Scheduling



For each sample

- 3 waves per main job
- 20k ~ 200k jobs per wave



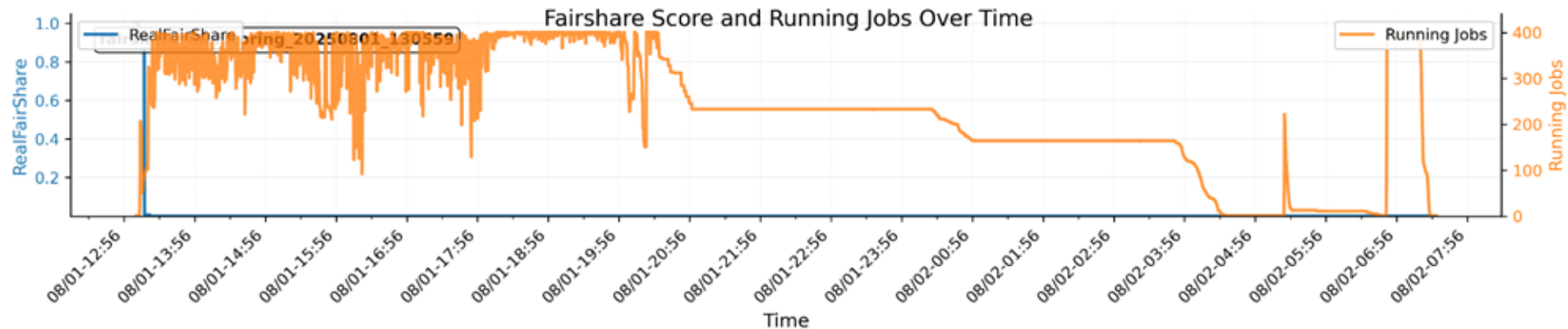
Challenge #1: Scheduling

Fairshare
score ▼

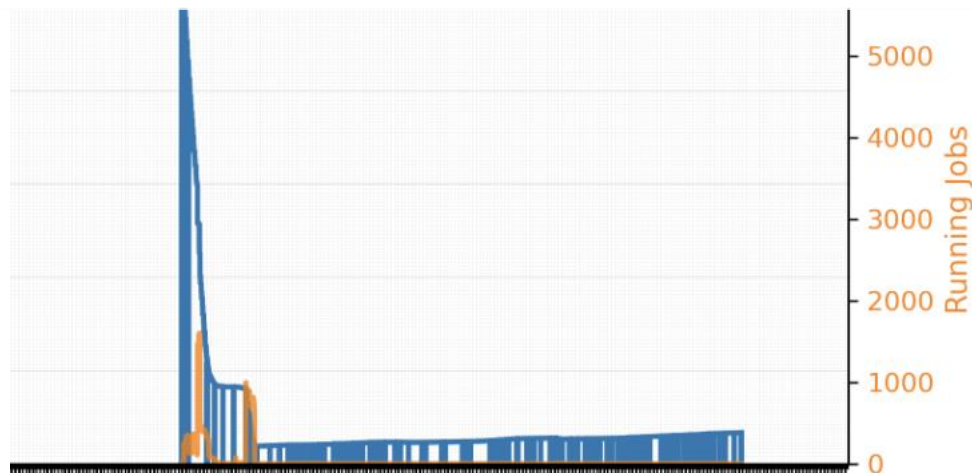
Waiting
time ▲

Total
time ▲

Main job
crashes



Challenge #1: Scheduling

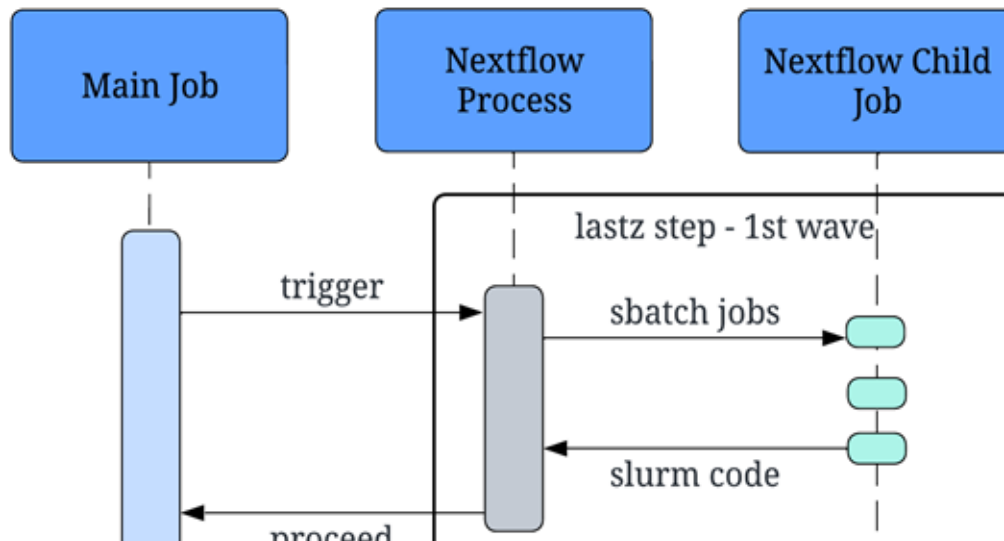


Additional Info:

Dynamic Fairshare, recovery

Challenge #2: Failure Tolerance

- Child job: failed (OOM, timeout, etc.)
 - Slurm: sends out an error signal
 - Nextflow calls off the job step
 - Nextflow calls off the entire workflow

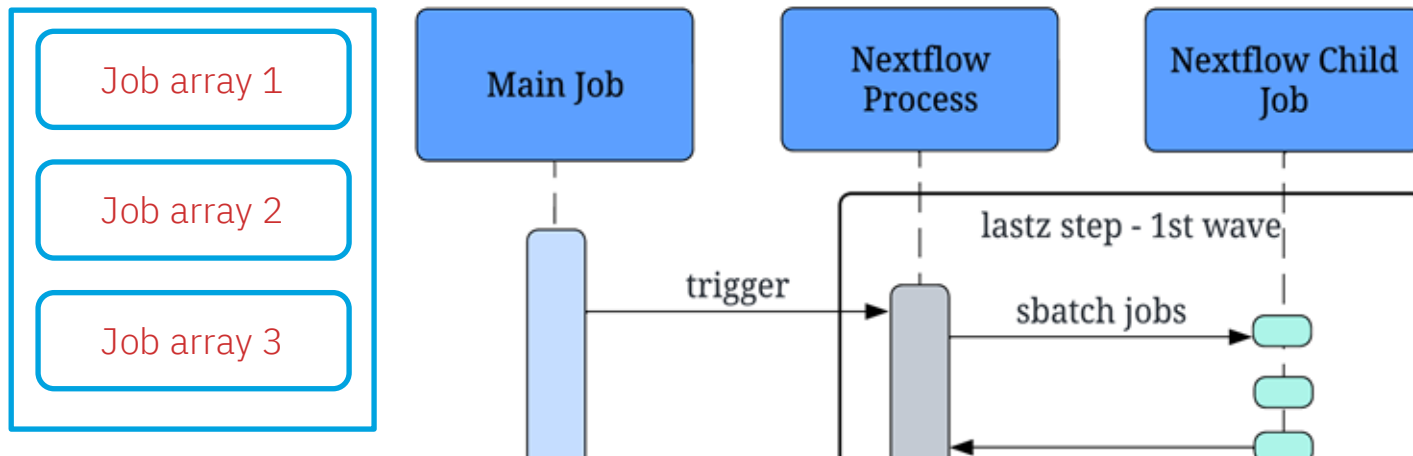


Challenge #3: Slurm RPC limit

- Hardcoded Slurm RPC limit
- Affected by other users as well
- Runaway child jobs – no logs, not in queue
 - Slurm log > nextflow log > child job log
- Main job stuck

Solutions / Best Practices

- Reduce child job size (time, memory, cores) for backfilling
- Retry mechanism, error handling
- Dynamic resource allocation & selection
- Enable the **Job Array** feature



Nextflow v25 Example

nextflow.config

```
params.samples = "samples/*.fq"
process TRIM {
    input:
        path fq
    output:
        path "${fq.baseName}.trim.fq" into trimmed_ch
    conda "/path/to/conda/envs/alignment"
    script:
        """
        fastp -i $fq -o ${fq.baseName}.trim.fq -q 20
        """
}
```

align.nf

```
#!/bin/bash
#SBATCH -c 1
...
module load nextflow

nextflow run main.nf -with-conda -with-mamba
```

main.sh

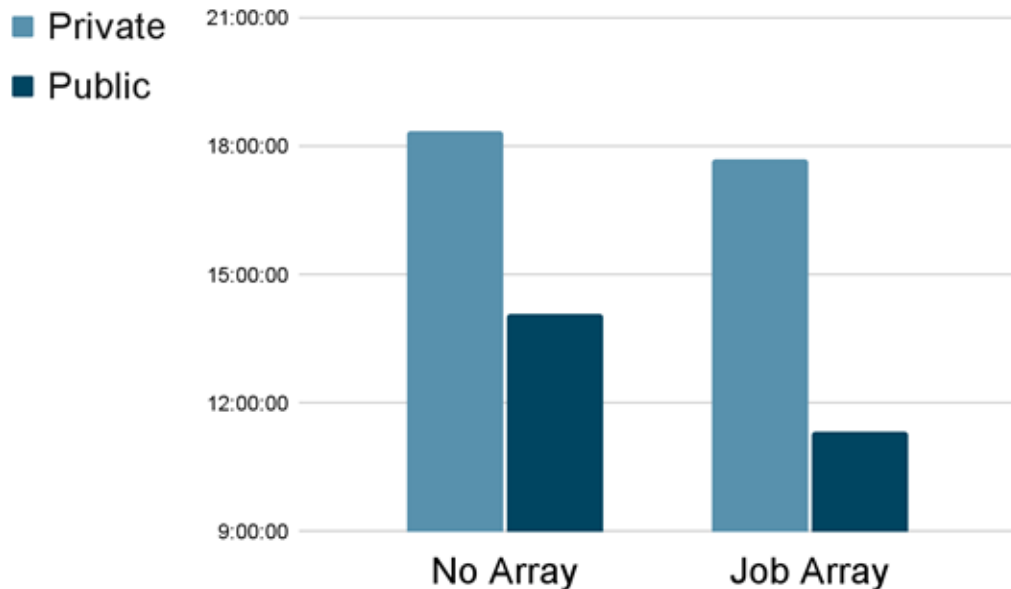
```
// Nextflow config for lastz jobs
executor {
    queueSize = 10000
    retry.maxAttempt = 3
    killBatchSize = 1000000
}

process {
    executor = 'slurm'
    memory = { 4.GB * task.attempt }
    time = { 1.hour * task.attempt }
    queue = 'public'
    cpus = 1
    array = 2000
    maxRetries = 5
    errorStrategy = 'retry'
    maxErrors = '-1'
    clusterOptions = '--signal=USR2@180'
}
```

Solutions / Best Practices

1. Reduce child job size (time, memory, cores) for backfilling
2. Retry mechanism, error handling
3. Dynamic resource allocation & selection
4. Enable the Job Array feature
5. Gracefully end the child jobs, adjust child job sizes
6. Limit child job counts, limit polling frequency
7. Out-of-band watchdog, post run assertion, **slurm tuning**
8. CPU-intensive: more cores vs. faster cores
9. Enable the Report feature: report.html, timeline.html
10. Refactor? Python-based -> Classic Nextflow -> nf-core lint

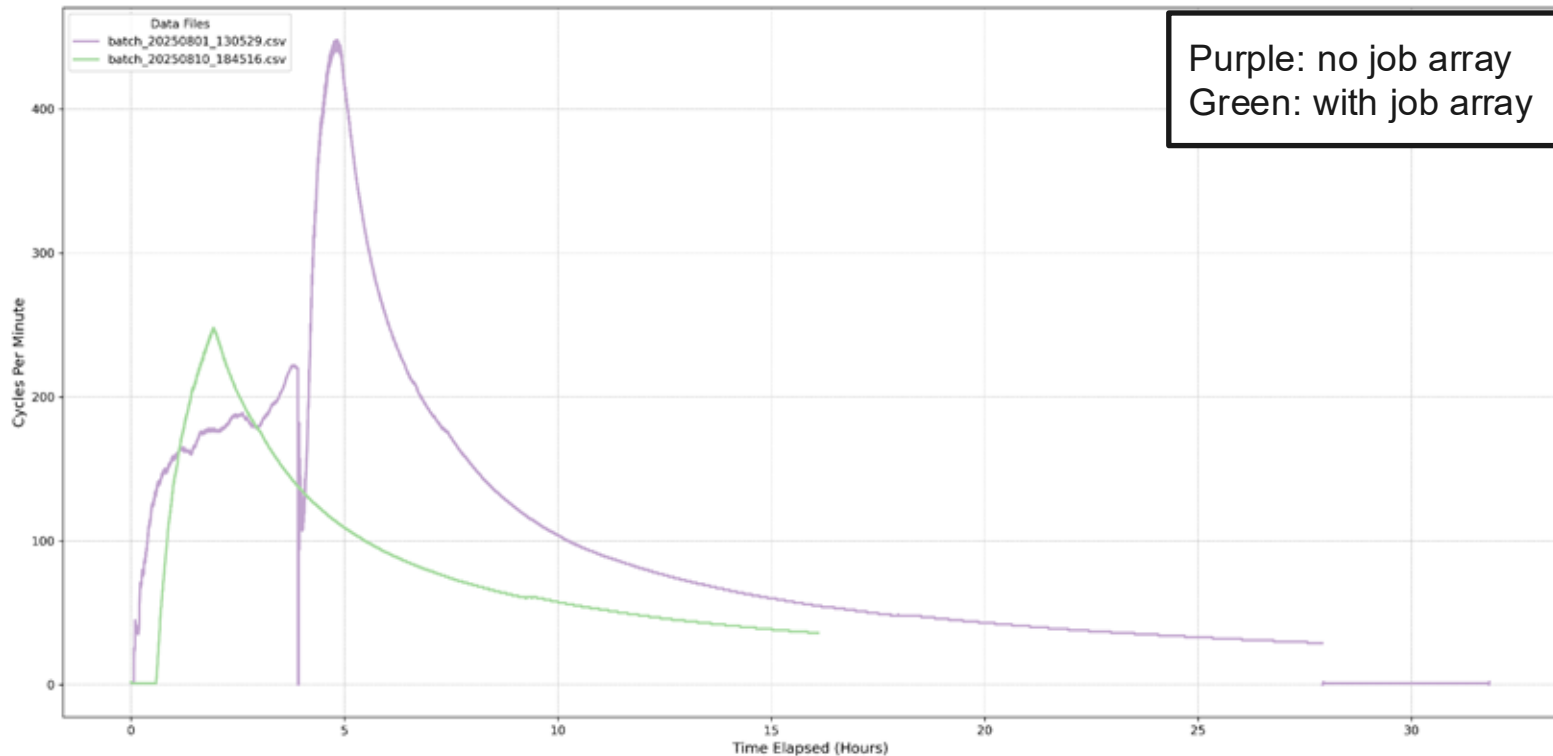
Benchmarking – Main Job Run Time



- Workstation:
 - 4 sample/ month
- HPC:
 - 30 samples/ month
- HPC & Job Array:
 - 90 samples/ month
- Nf-core refactor
 - ~110 samples/ month

Benchmarking – Slurm Stress

Slurm Performance Comparison



Real Example – slurm log

Pipeline completed successfully!

Final chain: /data/hlewin1/tianche5/amphibian_nf/Ascaphus_truei_Lissotriton_helveticus_chains/results/final/Ascaphus_truei.Lissotriton_helveticus.final.chain.gz

Run time : 1d 21h 39m 20s

executor > slurm (954524)

```
[49/5ffc28] MAK...O_TWO_BIT (Ascaphus_truei) | 1 of 1, cached: 1 ✓
[5d/9e580b] MAK..._BIT_INFO (Ascaphus_truei) | 1 of 1, cached: 1 ✓
[23/0df32b] MAK...T (Lissotriton_helveticus) | 1 of 1, cached: 1 ✓
[fa/15ce17] MAK...O (Lissotriton_helveticus) | 1 of 1, cached: 1 ✓
[b7/eb5bc7] MAK... (Ascaphus_truei (target)) | 1 of 1, cached: 1 ✓
[6c/a7512a] MAK...triton_helveticus (query)) | 1 of 1, cached: 1 ✓
[a8/392e98] MAK...4_1:1198800000-1208800000) | 951750 of 951750, retries: 1358 ✓
[da/8ff20c] MAK..._1_in_250000000_260000000) | 346 of 346, retries: 10 ✓
[99/8730a9] MAK...S:CHAIN_BUILD:PSL_SORT_ACC | 1 of 1, retries: 2 ✓
[d2/56be84] MAK...INS:CHAIN_BUILD:PSL_BUNDLE | 1 of 1, retries: 3 ✓
[fa/f77c50] MAK...D:AXT_CHAIN (bundle.3.psl) | 44 of 44, retries: 4 ✓
[0a/34416e] MAK...AIN_BUILD:CHAIN_MERGE_SORT | 1 of 1 ✓
[26/534b5e] MAK...AN_CHAINS:FILL_CHAIN_SPLIT | 1 of 1 ✓
[89/b0a490] MAK..._FILLER (infill_chain_953) | 1000 of 1000 ✓
[b4/fec2a3] MAK...AN_CHAINS:FILL_CHAIN_MERGE | 1 of 1 ✓
[a1/ba1cb7] MAK...elveticus.filled.chain.gz) | 1 of 1 ✓
[23/02ad40] MAK...leaned_intermediate.chain) | 1 of 1 ✓
```

Completed at: 08-Apr-2026 10:41:29

Duration : 1d 21h 47m 42s

CPU hours : 11'304.2 (0% cached, 1.5% failed)

Succeeded : 953'147

Cached : 6

Failed : 1'377

Take-away

- Nextflow would be a better choice
- Main job vs. Child job
- Scheduling & Failure Tolerance
- Retry with Dynamic resource allocation
- Job array support & tuning

Thank You!

ASU RTO Research Computing:
Nil Tianchen Mu · William Dizon

ASU RTO, Associate Director: Torey Battelle

ASU RTO Lead Scientific Software Eng.: Glen Otero



<https://nilablueshirt.github.io/WMS-on-HPC/>